

Pay2Peer

Plataforma segura de transferencias
y monedero digital

Memoria del Proyecto de Fin de Ciclo

Administración de Sistemas Informáticos en Red
Especialización en Ciberseguridad

Autor: Alejo Viñeta

Centro: IFP

Fase 1: 18 de diciembre de 2025

Fase 2: 5 de mayo de 2026

Versión: 3.0

Resumen

Pay2Peer es una aplicación web de tipo *fintech* que permite a sus usuarios registrarse, añadir fondos mediante tarjeta bancaria, transferir dinero entre pares (*peer-to-peer*), solicitar pagos y consultar su historial de movimientos. Su funcionamiento es análogo al de plataformas comerciales como PayPal o Bizum, pero ha sido diseñada íntegramente como proyecto académico para el ciclo de ASIR con especialización en Ciberseguridad.

El desarrollo se ha dividido en dos fases. La primera entregó un producto mínimo viable con autenticación, wallets, transferencias e integración con la pasarela de pagos Stripe. La segunda fase —objeto principal de esta memoria— abordó la securización completa del sistema según el marco STRIDE, el cumplimiento de la LOPD y el GDPR, la implementación de una cadena de bloques privada con algoritmo de consenso *Proof of Authority* para auditoría criptográfica, autenticación de dos factores TOTP, una suite de 149 pruebas automáticas *end-to-end* y el despliegue en producción sobre infraestructura propia con Docker, Nginx y Cloudflare.

El resultado es un sistema funcional, seguro y desplegado en un entorno real, que demuestra la aplicación práctica de los conocimientos adquiridos durante el ciclo formativo en las áreas de desarrollo web, administración de sistemas, redes y ciberseguridad.

Índice general

Resumen	1
1. Introducción	4
1.1. Contexto y motivación	4
1.2. Origen del proyecto	4
1.3. Estructura de la memoria	4
2. Objetivos	5
2.1. Objetivo general	5
2.2. Objetivos de la Fase 1	5
2.3. Objetivos de la Fase 2	5
3. Metodología	7
3.1. Herramientas de desarrollo	7
3.2. Análisis de seguridad	7
4. Arquitectura del sistema	8
4.1. Visión general	8
4.2. Frontend	8
4.3. Backend	9
4.4. Modelo de datos	9
5. Funcionalidades	10
5.1. Autenticación y gestión de sesiones	10
5.1.1. Registro	10
5.1.2. Inicio de sesión y tokens	10
5.1.3. Autenticación de dos factores (2FA)	10
5.2. Wallet y operaciones financieras	11
5.2.1. Wallet	11
5.2.2. Adición de fondos	11
5.2.3. Transferencias <i>peer-to-peer</i>	11
5.2.4. Solicitudes de dinero	11
5.2.5. Retiro de fondos	11
5.2.6. Historial de transacciones	12
5.3. Dashboard	12
6. Seguridad	13
6.1. Vulnerabilidades identificadas y remediaciones	13

6.2. Medidas complementarias	14
7. Cumplimiento legal (LOPD/GDPR)	15
7.1. Consentimiento informado	15
7.2. Derecho al olvido (Artículo 17 GDPR)	15
7.3. Portabilidad de datos (Artículo 20 GDPR)	15
7.4. Artículos cubiertos	16
8. Blockchain Proof of Authority	17
8.1. Propósito	17
8.2. Diseño y algoritmos	17
8.3. Ciclo de vida de un bloque	17
8.4. Verificación	18
9. Testing	19
9.1. Estrategia	19
9.2. Resultados	19
9.3. Aspectos técnicos relevantes	20
10.Despliegue e infraestructura	21
10.1. Stack de producción	21
10.2. Arquitectura Docker	21
10.3. Dominio y certificados TLS	22
10.4. Base de datos en la nube	22
10.5. Proceso de despliegue	22
11.Conclusiones	23
12.Trabajo futuro	24
Bibliografía	25

Capítulo 1

Introducción

1.1. Contexto y motivación

La digitalización de los servicios financieros es una de las tendencias más relevantes de la última década. Plataformas como PayPal, Revolut o Bizum han demostrado que los usuarios demandan soluciones de pago instantáneo, accesibles desde el navegador o el teléfono móvil, y con un nivel de seguridad equiparable al de la banca tradicional [17]. Sin embargo, detrás de la aparente sencillez de estas aplicaciones se esconde una complejidad técnica considerable: autenticación robusta, gestión de transacciones atómicas, cumplimiento normativo, protección frente a fraude y una infraestructura capaz de operar de forma continua.

Este proyecto nace con el objetivo de construir una plataforma de estas características desde cero, aplicando los conocimientos adquiridos durante el ciclo de ASIR con especialización en Ciberseguridad. No se trata de competir con productos comerciales, sino de demostrar que un estudiante con formación en administración de sistemas y seguridad informática es capaz de diseñar, implementar, securizar y desplegar un sistema financiero funcional.

1.2. Origen del proyecto

Pay2Peer tiene su origen en un proyecto de aprendizaje de React anterior al ciclo formativo. Aquel primer prototipo implementaba una interfaz básica de wallet con transferencias, pero carecía de cualquier medida de seguridad, infraestructura de despliegue o cumplimiento normativo. El ciclo de ASIR proporcionó la oportunidad —y los conocimientos— para transformar ese prototipo en un sistema completo.

1.3. Estructura de la memoria

Esta memoria se organiza de la siguiente forma: el Capítulo 2 define los objetivos del proyecto. El Capítulo 3 describe la metodología de trabajo. Los Capítulos 4 a 8 detallan el diseño técnico, las funcionalidades, la seguridad, el cumplimiento legal y la cadena de bloques. El Capítulo 9 presenta la estrategia de pruebas. El Capítulo 10 documenta la infraestructura de producción. Finalmente, los Capítulos 11 y 12 recogen las conclusiones y las líneas de trabajo futuro.

Capítulo 2

Objetivos

2.1. Objetivo general

Construir una aplicación *fintech* segura, conforme a la normativa europea de protección de datos, desplegada en un entorno de producción real y con trazabilidad criptográfica de todas las operaciones financieras.

2.2. Objetivos de la Fase 1

La primera fase se centró en obtener un producto mínimo viable:

F1.1 Registro e inicio de sesión con tokens JWT.

F1.2 Creación automática de wallet por usuario.

F1.3 Transferencias *peer-to-peer* entre usuarios registrados.

F1.4 Integración con Stripe para la adición de fondos mediante tarjeta bancaria.

F1.5 Historial de transacciones con detalle de emisor, receptor, importe y fecha.

F1.6 Solicitudes de dinero entre usuarios con flujo de aceptación o rechazo.

F1.7 Cola de mensajes con RabbitMQ para la gestión de picos de carga.

Todos los objetivos de esta fase fueron alcanzados y entregados en diciembre de 2025.

2.3. Objetivos de la Fase 2

La segunda fase perseguía llevar el sistema a un estado de producción seguro:

F2.1 Análisis de amenazas mediante el marco STRIDE y remediación de las 15 vulnerabilidades identificadas.

F2.2 Cumplimiento de la LOPD y el GDPR: consentimiento informado, derecho al olvido y portabilidad de datos.

F2.3 Implementación de una cadena de bloques privada con consenso *Proof of Authority* y firma digital Ed25519.

- F2.4 Autenticación de dos factores mediante TOTP (RFC 6238), compatible con Google Authenticator.
 - F2.5 *Rate limiting* en cuatro niveles: global, autenticación, recuperación de contraseña y pagos.
 - F2.6 Control de acceso basado en roles (RBAC): usuario, administrador y soporte.
 - F2.7 *Logging* estructurado con Winston para eventos de seguridad, transacciones y errores.
 - F2.8 Suite de pruebas *end-to-end*: 149 tests en 13 suites con Jest y Supertest.
 - F2.9 Despliegue en producción: servidor propio con Docker Compose, Nginx como proxy inverso y Cloudflare como CDN y terminador TLS.
 - F2.10 Integración con Stripe Connect para retiros a cuenta bancaria (implementación preparada para activación con cuenta empresarial).
 - F2.11 Migración de la base de datos a MongoDB Atlas para alta disponibilidad.
- Todos los objetivos han sido alcanzados.

Capítulo 3

Metodología

El desarrollo se ha llevado a cabo siguiendo una metodología iterativa e incremental, organizada en *sprints* informales de una a dos semanas de duración. Cada iteración producía una funcionalidad completa —backend, frontend y tests— antes de pasar a la siguiente.

3.1. Herramientas de desarrollo

- **Control de versiones:** Git con repositorio en GitHub.
- **Editor:** Visual Studio Code con extensiones para ESLint, Prettier y Tailwind CSS.
- **Entorno local:** Docker Compose para replicar el entorno de producción. MongoDB y RabbitMQ en contenedores.
- **Testing:** Jest como *runner*, Supertest para pruebas HTTP, MongoMemoryServer para base de datos en memoria.
- **Despliegue:** transferencia manual de archivos con `scp` y reconstrucción de contenedores con `docker compose build`.

3.2. Análisis de seguridad

La securización de la Fase 2 partió de un análisis STRIDE exhaustivo (documento de 709 líneas, incluido en el repositorio como `docs/threat-model-STRIDE.txt`). El análisis examinó cada componente del sistema —autenticación, API, base de datos, webhooks, infraestructura— y clasificó las amenazas en las seis categorías del modelo: *Spoofing*, *Tampering*, *Repudiation*, *Information Disclosure*, *Denial of Service* y *Elevation of Privilege*. Las 15 vulnerabilidades identificadas se priorizaron por criticidad y se remediaron en orden.

Capítulo 4

Arquitectura del sistema

4.1. Visión general

La arquitectura de Pay2Peer sigue el patrón clásico de tres capas —presentación, lógica de negocio y datos— desplegadas como microservicios en contenedores Docker. Todo el tráfico externo pasa por Cloudflare [4], que proporciona TLS, protección DDoS y caché de activos estáticos. Nginx [7] actúa como proxy inverso, enrutando las peticiones `/api/*` al backend y el resto al frontend.

Flujo de red en producción

```
1 Internet -> Cloudflare (CDN + HTTPS + WAF)
2           -> Nginx (reverse proxy, puerto 80)
3               |-> Frontend (Next.js, puerto 3000)
4               |-> Backend API (Express, puerto 5000)
5                   |-> MongoDB Atlas (cloud)
6                   |-> RabbitMQ (puerto 5672)
```

4.2. Frontend

El frontend está construido con **Next.js 15** (React 19) y TypeScript [26]. Se eligió Next.js por su capacidad de renderizado híbrido (servidor y cliente), su sistema de rutas basado en el sistema de archivos y la facilidad de despliegue como aplicación *standalone* en un contenedor Docker [6].

- **Estilos:** Tailwind CSS [24], que elimina los conflictos de cascada habituales en aplicaciones SPA y reduce el tamaño del *bundle* al incluir solo las clases utilizadas.
- **Gráficos:** Recharts [20] para los histogramas del dashboard.
- **Pagos:** Stripe Elements [22] mediante `react-stripe-js`, que gestiona los datos de tarjeta sin que pasen por el servidor del proyecto (cumplimiento PCI DSS [18]).
- **Gestión de estado:** React Context (`AuthContext`) para autenticación, con JWT [1] en memoria y *refresh token* con rotación automática.

4.3. Backend

El backend es una API REST construida con **Express.js 5** [16] sobre Node.js 22. Se eligió Express por su madurez, ecosistema de *middleware* y compatibilidad con las librerías de Stripe y JWT.

- **Base de datos:** MongoDB [15] con Mongoose como ODM. La elección de MongoDB se justifica por la flexibilidad que ofrece durante un desarrollo iterativo donde el esquema evoluciona frecuentemente. En mayo de 2026 se migró a **MongoDB Atlas** [14] (cloud) para garantizar disponibilidad y facilitar la arquitectura de *failover*.
- **Autenticación:** `express-jwt` para la validación de tokens [1]; `bcrypt` [19] con `EksBlowfish` para el hash de contraseñas; *refresh tokens* almacenados en base de datos con rotación en cada uso.
- **Validación:** `express-validator` en todos los endpoints que reciben datos de usuario.
- **Pagos:** Stripe API [22] con `PaymentIntents` para depósitos y verificación de firma en webhooks.
- **Email:** SendGrid [25] para notificaciones transaccionales (registro, transferencias, disputas, recuperación de contraseña).
- **Cola de mensajes:** RabbitMQ [3] como *buffer* para la gestión de picos de carga.
- **Logging:** Winston [8] con niveles estructurados y persistencia en archivos rotativos.
- **Blockchain:** cadena privada PoA integrada, con persistencia en MongoDB.

4.4. Modelo de datos

La base de datos se organiza en las colecciones descritas en la Tabla 4.1.

Tabla 4.1: Colecciones principales de la base de datos.

Colección	Campos clave
<code>users</code>	name, email, password (bcrypt), role, twoFactorSecret, twoFactorEnabled, stripeAccountId
<code>wallets</code>	author (ref User), funds (formateado en EUR), frozen, paymentMethod
<code>transactions</code>	sender, receiver, amount, stripeSender, date
<code>requestmoney</code>	sender, receiver, amount, status (pending/accepted/rejected/cancelled), date
<code>blocks</code>	index, hash, previousHash, transactions, signature, timestamp
<code>userconsents</code>	userId, privacyPolicyAccepted, termsOfServiceAccepted, ipAddress, userAgent, date
<code>resettokens</code>	userId, token (hash SHA-256), expiresAt (TTL 1 hora)

Capítulo 5

Funcionalidades

5.1. Autenticación y gestión de sesiones

5.1.1. Registro

El registro requiere nombre, apellido, correo electrónico y contraseña. La contraseña se valida contra criterios de complejidad y se almacena hasheada con `bcrypt` [19] (10 rondas de sal). El registro exige la aceptación explícita de la política de privacidad y los términos de servicio, registrándose el consentimiento con la IP y el *user-agent* del cliente (véase el Capítulo 7).

La respuesta del endpoint de registro es deliberadamente genérica —"**Account created or already exists**"— para impedir la enumeración de usuarios mediante fuerza bruta sobre el campo de correo electrónico.

5.1.2. Inicio de sesión y tokens

El inicio de sesión devuelve un *access token* JWT con expiración de 15 minutos y un *refresh token* de 7 días almacenado en la base de datos. El *refresh token* implementa rotación: cada uso genera un nuevo par de tokens e invalida el anterior, limitando la ventana de exposición en caso de robo.

El rol del usuario no se almacena en el token JWT sino que se consulta en la base de datos en cada operación que lo requiere, evitando escaladas de privilegio mediante manipulación del payload del token.

5.1.3. Autenticación de dos factores (2FA)

Se implementó TOTP conforme a la RFC 6238 [11], compatible con Google Authenticator, Authy y aplicaciones similares. La implementación utiliza exclusivamente el módulo `crypto` nativo de Node.js, evitando la dependencia `otplib` que presentaba incompatibilidades con el sistema de módulos ESM de Jest.

El flujo de 2FA es el siguiente:

1. **Activación:** el usuario genera un secreto TOTP desde la página de ajustes. El backend devuelve el secreto codificado como URI `otpauth://` y como código QR en

formato *data URL*.

2. **Verificación:** el usuario escanea el QR con su aplicación de autenticación e introduce el código de 6 dígitos. Si es correcto, el 2FA queda activado en su cuenta.
3. **Login:** cuando un usuario con 2FA activo introduce sus credenciales, el sistema devuelve un `tempToken` (JWT con TTL de 5 minutos y flag `pending2fa: true`) en lugar del token de acceso definitivo. El cliente debe enviar el código TOTP para completar la autenticación.

5.2. Wallet y operaciones financieras

5.2.1. Wallet

Cada usuario dispone de una wallet creada automáticamente en el momento del registro con saldo inicial de 0. Los importes se gestionan internamente con la librería `currency.js` [5] para evitar errores de aritmética en punto flotante.

5.2.2. Adición de fondos

El usuario puede añadir fondos mediante tarjeta bancaria a través de Stripe Elements [22]. El flujo es:

1. El frontend crea un `PaymentIntent` a través del backend.
2. Stripe Elements captura los datos de tarjeta sin que pasen por el servidor (cumplimiento PCI DSS).
3. El frontend confirma el pago y solicita al backend la verificación mediante `stripe.paymentIntents`.
4. El backend acredita el importe en la wallet del usuario y registra la transacción.

5.2.3. Transferencias *peer-to-peer*

El usuario introduce el correo electrónico del destinatario y el importe. El backend verifica que el emisor dispone de fondos suficientes, debita la wallet emisora, acredita la wallet receptora y registra la transacción tanto en MongoDB como en la cadena de bloques.

5.2.4. Solicitudes de dinero

Un usuario puede solicitar dinero a otro. La solicitud queda en estado `pending` hasta que el receptor la acepta (ejecutando automáticamente la transferencia) o la rechaza. El emisor de la solicitud puede cancelarla mientras esté pendiente. Se envían notificaciones por correo electrónico en cada cambio de estado.

5.2.5. Retiro de fondos

La funcionalidad de retiro permite debitar el saldo de la wallet. Para el entorno de demostración se utiliza un retiro simulado que registra la operación sin ejecutar un payout bancario real. Paralelamente, se ha implementado la infraestructura de Stripe Connect [23]

(Express Accounts) que permite retiros reales a cuenta bancaria una vez se disponga de una cuenta empresarial verificada en Stripe.

5.2.6. Historial de transacciones

El historial muestra todas las operaciones del usuario (ingresos y gastos) con paginación de 10 elementos por página y filtrado por tipo. Cada transacción incluye fecha, contraparte, tipo e importe.

5.3. Dashboard

El dashboard ofrece una vista consolidada del estado del usuario: saldo actual, histograma de transacciones de los últimos 7 días (ingresos frente a gastos), transacciones recientes y solicitudes de dinero pendientes de respuesta.

Capítulo 6

Seguridad

La seguridad de la Fase 2 se abordó de forma sistemática a partir de un análisis de amenazas basado en el modelo STRIDE de Microsoft [13]. El análisis, documentado en un informe de 709 líneas disponible en el repositorio (`docs/threat-model-STRIDE.txt`), identificó 15 vulnerabilidades clasificadas por categoría y criticidad. Todas han sido remediadas.

6.1. Vulnerabilidades identificadas y remediaciones

La Tabla 6.1 recoge las 15 vulnerabilidades con su categoría STRIDE, descripción y solución aplicada.

Tabla 6.1: Vulnerabilidades identificadas en el análisis STRIDE.

ID	Categoría	Descripción	Solución
S-01	Spoofing	JWT sin expiración adecuada	Access token 15 min + refresh token 7 d con rotación
S-02	Spoofing	Refresh token sin rotación	Rotación automática en cada uso; invalidación del anterior
S-03	Spoofing	Rol del usuario leído del token	El rol se consulta en base de datos en cada operación
S-04	Tampering	Webhooks de Stripe sin validación	Verificación de firma con <code>constructEvent()</code>
T-03	Tampering	<i>Mass assignment</i> en wallet	<i>Deconstructing</i> explícito en controladores
T-04	Tampering	<i>Mass assignment</i> en usuario	<i>Deconstructing</i> explícito en controladores
E-01	Elevation	Endpoints con ID en URL	Sustituidos por <code>/me</code> (dato del JWT)
E-02	Elevation	Wallet con saldo negativo	Verificación de saldo $\geq$ importe; init a 0
E-03	Elevation	Sin control de roles	Middleware <code>requireRole</code> con RBAC

ID	Categoría	Descripción	Solución
I-01	Info Discl.	Credenciales RabbitMQ en código	Variables de entorno
I-03	Info Discl.	<i>Stack traces</i> en errores 500	Middleware global de errores
I-04	Info Discl.	Enumeración de emails en registro	Respuesta genérica
I-05	Info Discl.	Hash de contraseña en JSON	Transform <code>toJSON</code> elimina campos sensibles
I-06	Info Discl.	Blockchain pública sin auth	Protección JWT
D-01	DoS	Sin <i>rate limiting</i>	4 niveles (global, auth, password, pagos)
R-01	Repudiation	Sin logging de seguridad	Winston con eventos tipados

6.2. Medidas complementarias

Además de las remediaciones anteriores, se aplicaron las siguientes medidas de endurecimiento:

- **Cabecera `x-powered-by` deshabilitada** para no revelar la tecnología del servidor.
- **Límite de tamaño del cuerpo HTTP** a 100 KB (`express.json({limit: "100kb"})`) para prevenir ataques de *payload* masivo.
- **Congelación de wallets** ante disputas de Stripe: cuando se recibe un webhook `charge.dispute.created`, la wallet implicada se congela automáticamente, bloqueando transferencias y retiros hasta la resolución.
- **Cabeceras de seguridad en Nginx:** `X-Frame-Options`, `X-Content-Type-Options`, `X-XSS-Protection` y `Referrer-Policy`.

Capítulo 7

Cumplimiento legal (LOPD/GDPR)

La aplicación implementa los requisitos de la Ley Orgánica de Protección de Datos (LOPD) [10] española y del Reglamento General de Protección de Datos (GDPR) [21] de la Unión Europea que resultan aplicables a un proyecto de estas características.

7.1. Consentimiento informado

El registro requiere la aceptación obligatoria de la política de privacidad y los términos de servicio. El backend rechaza el alta con HTTP 400 si cualquiera de los dos campos es `false`. El consentimiento se almacena en la colección `userconsents` con la fecha, la dirección IP del cliente y el *user-agent* del navegador, cumpliendo los requisitos probatorios del Artículo 7 del GDPR.

7.2. Derecho al olvido (Artículo 17 GDPR)

El endpoint DELETE `/api/users/me` ejecuta un borrado en cascada:

1. Elimina los registros de consentimiento.
2. Revoca todos los *refresh tokens* activos (cierra todas las sesiones).
3. Elimina las wallets del usuario.
4. Anonimiza las transacciones: los campos `sender` y `receiver` se establecen a `null`, conservando el importe para mantener la integridad de la cadena de bloques.
5. Elimina las solicitudes de dinero.
6. Elimina la cuenta de usuario.

Las entradas de la cadena de bloques no se eliminan porque la integridad de la cadena —cada bloque referencia el hash del anterior— es un requisito de auditoría.

7.3. Portabilidad de datos (Artículo 20 GDPR)

El endpoint GET `/api/users/me/export` devuelve un documento JSON con toda la información del usuario: perfil, consentimientos, wallets, transacciones y solicitudes de

dinero.

7.4. Artículos cubiertos

La Tabla 7.1 resume los artículos del GDPR cubiertos y su implementación.

Tabla 7.1: Artículos del GDPR cubiertos.

Artículo	Descripción	Estado
Art. 5	Minimización de datos	Implementado
Art. 7	Consentimiento verificable	Implementado
Art. 9	Seguridad del tratamiento	Implementado
Art. 12	Auditoría	Implementado
Art. 15	Derecho de acceso	Implementado
Art. 17	Derecho al olvido	Implementado
Art. 20	Portabilidad	Implementado
Art. 25	Privacidad por diseño	Implementado
Art. 32	Medidas técnicas de seguridad	Implementado

Capítulo 8

Blockchain Proof of Authority

8.1. Propósito

La cadena de bloques de Pay2Peer no es una blockchain pública ni está orientada a criptomonedas. Es un **registro de auditoría criptográficamente verificable** que complementa la base de datos relacional. Su función es garantizar que ninguna transacción pueda ser alterada retrospectivamente sin que la alteración sea detectable.

8.2. Diseño y algoritmos

- **Hash de bloque:** SHA-256 calculado sobre la concatenación del índice, hash del bloque anterior, *timestamp*, transacciones y *nonce*.
- **Firma digital:** Ed25519 [2] (*Proof of Authority*). El servidor firma cada bloque con su clave privada; la clave pública se registra como autoridad reconocida. Solo las autoridades registradas pueden sellar bloques.
- **Prueba de trabajo:** dificultad configurable (el hash debe comenzar con N ceros). En producción se utiliza dificultad 2 para equilibrar velocidad y seguridad.
- **Bloque génesis:** índice 0, `previousHash` = "0", generado automáticamente si la colección `blocks` está vacía.

8.3. Ciclo de vida de un bloque

1. Cada transacción (P2P o Stripe) se añade al *pool* de transacciones pendientes.
2. Un temporizador intenta sellar un bloque cada 10 segundos, o inmediatamente si el *pool* alcanza 10 transacciones.
3. Se construye el bloque, se mina (PoW) y se firma (PoA).
4. El bloque se persiste en MongoDB.
5. El *pool* se vacía.

8.4. Verificación

El endpoint `GET /api/blockchain/verify` recorre toda la cadena recalculando los hashes y verificando las firmas Ed25519 de cada bloque. Si la cadena es íntegra, devuelve `{valid:true}`. Si se detecta una inconsistencia, describe el bloque corrupto y la naturaleza de la discrepancia.

Capítulo 9

Testing

9.1. Estrategia

Se optó por una suite de pruebas *end-to-end* (E2E) que ejercita el sistema completo: servidor HTTP, lógica de negocio y base de datos [9], [12]. Cada ejecución lanza el servidor real contra una instancia de MongoMemoryServer (MongoDB en memoria), garantizando aislamiento y reproducibilidad sin necesidad de una base de datos externa.

Los tests de Stripe utilizan un cliente *mock* inyectable mediante la función `_setStripeForTesting()`, que permite simular respuestas de la API de Stripe sin realizar llamadas reales.

9.2. Resultados

La suite contiene **149 pruebas** distribuidas en **13 archivos de test**, con una tasa de éxito del 100 %.

Tabla 9.1: Distribución de tests por archivo.

Archivo	Cobertura funcional	Tests
01-auth	Registro, login, refresh, logout	15
02-wallet	Balance, fondos, operaciones	10
03-transactions	Transferencias P2P, paginación, historial	12
04-token-lifecycle	Expiración, rotación, revocación de tokens	8
05-security	OWASP Top 10, mass assignment, enumeración	15
06-rate-limiting	Los 4 niveles de rate limiting	10
07-stripe-webhooks	Verificación de firma, prevención de tampering	6
08-rbac	Control de acceso por rol	8
09-lopd	Consentimiento, olvido, exportación	12
10-blockchain	Verificación de cadena, sellado, integridad	10
11-new-features	Solicitudes, paginación, retiro, freeze	15
12-2fa	Setup TOTP, verificación, login flow, disable	9
13-stripe-connect	Onboarding, status, retiro real, rollback	15
Total		149

9.3. Aspectos técnicos relevantes

- El *rate limiter* se resetea en cada `beforeEach` mediante `resetAllStores()` para evitar falsos HTTP 429 entre tests consecutivos.
- La implementación TOTP en los tests utiliza `crypto` nativo (RFC 6238) en lugar de `otplib`, que presentaba incompatibilidades con el sistema de módulos de Jest.
- Los tests de rollback de Stripe Connect verifican que, si la transferencia a la cuenta conectada falla, el saldo de la wallet se restaura al valor original.

Capítulo 10

Despliegue e infraestructura

10.1. Stack de producción

Tabla 10.1: Componentes del entorno de producción.

Componente	Tecnología	Versión
Servidor	VM propia (Linux)	Debian 12
CDN / HTTPS / WAF	Cloudflare	Free plan
Proxy inverso	Nginx	Alpine
Orquestación	Docker Compose	v2
Frontend	Next.js (standalone)	15
Backend	Express.js	5
Runtime	Node.js	22 (Alpine)
Base de datos	MongoDB Atlas	7.0
Cola de mensajes	RabbitMQ	3 (Alpine)

10.2. Arquitectura Docker

Los servicios se organizan en una red interna Docker (`pay2peer`). Únicamente Nginx expone puertos al exterior (80). El backend se ejecuta con el sistema de archivos en modo de solo lectura (`read_only: true`) con directorios temporales montados en `tmpfs` para `/tmp` y `/app/logs`.

Topología de servicios

```
1  nginx:80    ->  frontend:3000  (Next.js standalone)
2                ->  backend:5000   (Express API)
3                ->  MongoDB Atlas (cloud, puerto 27017/TLS)
4                ->  rabbitmq:5672 (interno)
```

Las variables de entorno sensibles (claves de Stripe, secreto JWT, credenciales de base de datos) se gestionan mediante un archivo `.env.production` excluido del control de versiones.

10.3. Dominio y certificados TLS

- **Dominio:** `www.pauoscaralejo.es`
- **TLS:** gestionado por Cloudflare con certificado automático en modo *Full (Strict)*.
- **DNS:** registros A apuntando al servidor; el tráfico se proxea a través de Cloudflare.

10.4. Base de datos en la nube

En mayo de 2026 se migró la base de datos desde el contenedor MongoDB local a **MongoDB Atlas** (cluster gratuito M0, 512 MB). La migración se realizó mediante `mongodump/mongorestore` sin pérdida de datos. Esta decisión responde a dos objetivos:

1. **Disponibilidad:** Atlas proporciona réplicas automáticas y backups diarios, eliminando el riesgo de pérdida de datos ante un fallo del servidor.
2. **Facilitar el *failover*:** al externalizar la base de datos, múltiples servidores de aplicación pueden compartir el mismo estado, requisito previo para la arquitectura de alta disponibilidad descrita en el Capítulo 12.

10.5. Proceso de despliegue

El despliegue es manual: los archivos modificados se transfieren al servidor mediante `scp` y se reconstruyen los contenedores afectados con `docker compose build`. Aunque funcional, este proceso es susceptible de error humano y se plantea su automatización como trabajo futuro.

Capítulo 11

Conclusiones

La Fase 2 del proyecto Pay2Peer ha cumplido satisfactoriamente todos los objetivos planteados. Se ha transformado un prototipo funcional pero inseguro en una aplicación que:

- Ha sido analizada sistemáticamente con el modelo STRIDE, identificando y remediando 15 vulnerabilidades de seguridad.
- Cumple con los requisitos aplicables de la LOPD y el GDPR: consentimiento informado, derecho al olvido, portabilidad de datos y privacidad por diseño.
- Dispone de un registro de auditoría criptográficamente verificable basado en una cadena de bloques privada con firma digital Ed25519.
- Implementa autenticación de dos factores TOTP, compatible con las aplicaciones de autenticación más extendidas.
- Está respaldada por 149 pruebas automáticas *end-to-end* que cubren la funcionalidad, la seguridad, el cumplimiento legal y la integración con servicios externos.
- Está desplegada en un entorno de producción real con dominio propio, TLS y CDN.

El aprendizaje más relevante del proyecto ha sido comprobar que la seguridad aplicada es un proceso iterativo: analizar, priorizar, remediar y verificar. Muchas de las vulnerabilidades más habituales —*mass assignment*, ausencia de *rate limiting*, tokens sin expiración, webhooks sin verificar— requieren pocas líneas de código para su corrección, pero exigen saber qué buscar y dónde buscar.

Un aprendizaje secundario, pero igualmente valioso, ha sido la gestión de la infraestructura de producción: comprender cómo interactúan Docker, Nginx, Cloudflare y los certificados TLS, y cómo diagnosticar problemas en un entorno donde el desarrollador no tiene acceso a la consola del navegador del usuario.

Capítulo 12

Trabajo futuro

1. **Alta disponibilidad:** despliegue de un segundo nodo de aplicación en un servidor doméstico conectado mediante Cloudflare Tunnel, con *failover* automático gestionado por Cloudflare y base de datos compartida en MongoDB Atlas.
2. **Retiros bancarios reales:** activación de Stripe Connect (Express Accounts) con cuenta empresarial verificada. La infraestructura de backend y frontend ya está implementada y probada.
3. **CI/CD:** implementación de un *pipeline* con GitHub Actions que ejecute la suite de tests en cada *push* y despliegue automáticamente a producción si todas las pruebas pasan.
4. **Migración a PostgreSQL:** sustitución de MongoDB por PostgreSQL con Prisma como ORM, aprovechando que el esquema está estabilizado y se beneficiaría de un sistema de tipos más estricto y transacciones ACID nativas.
5. **SSO/OAuth:** integración de inicio de sesión con Google o GitHub para delegar la gestión de credenciales a proveedores de identidad consolidados.
6. **Páginas legales:** creación de páginas estáticas en el frontend (`/privacy`, `/terms`) que complementen el consentimiento registrado en el backend.
7. **Notificación de brechas (Art. 33 GDPR):** mecanismo de alerta al responsable de protección de datos ante accesos no autorizados detectados por el sistema de *logging*.

Bibliografía

- [1] Auth0. «Introduction to JSON Web Tokens,» visitado 30 de abr. de 2026. dirección: <https://jwt.io/introduction>.
- [2] D. J. Bernstein, N. Duif, T. Lange, P. Schwabe y B.-Y. Yang. «High-speed high-security signatures. »dirección: <https://ed25519.cr.yp.to/>.
- [3] Broadcom, Inc. «RabbitMQ Documentation,» visitado 30 de abr. de 2026. dirección: <https://www.rabbitmq.com/docs>.
- [4] Cloudflare, Inc. «Cloudflare Documentation,» visitado 30 de abr. de 2026. dirección: <https://developers.cloudflare.com>.
- [5] S. Dennedy. «currency.js — Lightweight library for working with currency values,» visitado 30 de abr. de 2026. dirección: <https://currency.js.org>.
- [6] Docker, Inc. «Docker Documentation,» visitado 30 de abr. de 2026. dirección: <https://docs.docker.com>.
- [7] F5, Inc. «NGINX Documentation,» visitado 30 de abr. de 2026. dirección: <https://nginx.org/en/docs/>.
- [8] Flatiron. «winston — A logger for just about everything,» visitado 30 de abr. de 2026. dirección: <https://github.com/winstonjs/winston>.
- [9] Ladjs. «SuperTest — HTTP assertions made easy,» visitado 30 de abr. de 2026. dirección: <https://github.com/ladjs/supertest>.
- [10] *Ley Orgánica 3/2018, de Protección de Datos Personales y garantía de los derechos digitales (LOPDGDD)*, 5 de dic. de 2018. dirección: <https://www.boe.es/eli/es/1o/2018/12/05/3>.
- [11] D. M’Raihi, S. Machani, M. Pei y J. Rydell, «TOTP: Time-Based One-Time Password Algorithm,» IETF, RFC 6238, 2011. dirección: <https://datatracker.ietf.org/doc/html/rfc6238>.
- [12] Meta Platforms, Inc. «Jest — Delightful JavaScript Testing,» visitado 30 de abr. de 2026. dirección: <https://jestjs.io>.
- [13] Microsoft. «The STRIDE Threat Model,» visitado 30 de abr. de 2026. dirección: <https://learn.microsoft.com/en-us/azure/security/develop/threat-modeling-tool-threats>.
- [14] MongoDB, Inc. «MongoDB Atlas — Cloud Database Service,» visitado 30 de abr. de 2026. dirección: <https://www.mongodb.com/atlas>.
- [15] MongoDB, Inc. «MongoDB Documentation,» visitado 30 de abr. de 2026. dirección: <https://www.mongodb.com/docs/>.

- [16] OpenJS Foundation. «Express.js — Fast, unopinionated, minimalist web framework for Node.js,» visitado 30 de abr. de 2026. dirección: <https://expressjs.com>.
- [17] OWASP Foundation. «OWASP Top Ten — 2021,» visitado 30 de abr. de 2026. dirección: <https://owasp.org/www-project-top-ten/>.
- [18] PCI Security Standards Council. «PCI DSS Quick Reference Guide,» visitado 30 de abr. de 2026. dirección: <https://www.pcisecuritystandards.org>.
- [19] N. Provos y D. Mazières. «A Future-Adaptable Password Scheme. »dirección: <https://www.usenix.org/legacy/events/usenix99/provos/provos.pdf>.
- [20] Recharts Group. «Recharts — A composable charting library for React,» visitado 30 de abr. de 2026. dirección: <https://recharts.org>.
- [21] *Reglamento (UE) 2016/679 del Parlamento Europeo y del Consejo (RGPD/GDPR)*, 27 de abr. de 2016. dirección: <https://eur-lex.europa.eu/eli/reg/2016/679/oj>.
- [22] Stripe, Inc. «Stripe API Reference,» visitado 30 de abr. de 2026. dirección: <https://stripe.com/docs/api>.
- [23] Stripe, Inc. «Stripe Connect — Platforms and Marketplaces,» visitado 30 de abr. de 2026. dirección: <https://stripe.com/docs/connect>.
- [24] Tailwind Labs. «Tailwind CSS Documentation,» visitado 30 de abr. de 2026. dirección: <https://tailwindcss.com/docs>.
- [25] Twilio. «SendGrid Email API Documentation,» visitado 30 de abr. de 2026. dirección: <https://docs.sendgrid.com>.
- [26] Vercel. «Next.js — The React Framework,» visitado 30 de abr. de 2026. dirección: <https://nextjs.org/docs>.